

AWS REST API Lab Part 1: Create REST API and Define Resources

To Begin with the Lab

Summary of the Lab

In this part of the lab, a **REST API** was created in **API Gateway** to expose the backend built in the earlier prerequisites. The lab reused the existing **DynamoDB** table named **UsersTable** and the three **AWS Lambda** functions from the previous HTTP API workflow, so only the API layer changed from HTTP API to REST API. A new REST API named **UserRestAPI** was created with a regional endpoint, a TLS 1.3/1.2-compatible security policy, basic endpoint access mode, and IPv4 addressing. After that, the required resources were defined so the REST API could support separate paths for creating a user and working with a specific `userId`. The lab ended by preparing the API for method attachment in the next part.

Understanding REST API Resources

A **REST API** in **API Gateway** is built around resource paths, and each unique path must be defined before methods are attached. This exists so the API can clearly represent different operations, such as working with the full user collection or targeting a single user by ID. In this lab, `/user` represents the collection-level resource, while `/user/{userId}` represents a single record. That structure makes it easier to connect POST, GET, and DELETE methods to the correct Lambda function later.

Example:

A frontend app wants to create a new user, fetch a specific user, and delete that same user later. It will use `/user` for creating the record and `/user/{userId}` for reading or deleting one user by ID. This resource structure keeps the REST API organized and makes the request routing simple.

Lab Steps

Step 1 – Open API Gateway

- Sign in to the AWS Management Console.
- Open **API Gateway**.
- Click **Create API**.

API Gateway

APIs

Custom domain names
Domain name access associations
VPC links
AgentCore targets

Usage plans
API keys
Client certificates
Settings

Developer portals

Portals
Portal products

APIs (14/14)

Find APIs

	Name	Description	ID	Protocol	API endpoint type	Created	Security policy	API status
☐	api-intelcore1	for testing	api-intelcore1	REST	Regional	2026-05-16	TLS_1_0	Available
☐	APIControl1	for demo	api-control1	REST	Regional	2026-01-07	TLS_1_0	Available
☐	APIControl2	for demo	api-control2	REST	Regional	2026-01-07	TLS_1_0	Available
☐	apigateway2-API	Created by AWS Lambda	api-gateway2-API	REST	Regional	2025-02-19	TLS_1_0	Available
☐	apigateway3		api-gateway3	REST	Regional	2025-02-19	TLS_1_0	Available
☐	apigateway4		api-gateway4	REST	Regional	2025-02-19	TLS_1_0	Available
☐	awsapigateway1-API	Created by AWS Lambda	api-gateway1-API	REST	Regional	2025-02-19	TLS_1_0	Available
☐	awslambdap1-API	Created by AWS Lambda	api-lambda1-API	REST	Regional	2026-05-16	TLS_1_0	Available
☐	demoapi1	demoapi	api-demo1	REST	Regional	2025-04-28	TLS_1_0	Available
☐	IntelCoreWebAPI2	for demo	api-intelcore2	REST	Regional	2026-05-16	TLS_1_0	Available
☐	mydemo1		api-demo1	REST	Regional	2025-04-28	TLS_1_0	Available

Created by AWS

- Select **REST API**.
- Click **Build**.

Create API

Build low-latency and cost-effective REST APIs with built-in features such as OIDC and OAuth2, and native CORS support.

Works with the following:
Lambda, HTTP backends

WebSocket API

Build a WebSocket API using persistent connections for real-time use cases such as chat applications or dashboards.

Works with the following:
Lambda, HTTP, AWS Services

REST API

Develop a REST API where you gain complete control over the request and response along with API management capabilities.

Works with the following:
Lambda, HTTP, AWS Services

Step 2 – Create the REST API

- Select **New API**.
- Enter the API name: **UserRestAPI**
- Choose the endpoint type as **Regional**.
- Select the security policy that supports TLS 1.3 and TLS 1.2.
- Set endpoint access mode to **Basic**.

API details

☒ **New API**
Create a new REST API.

☐ **Clone existing API**
Create a copy of an API in this AWS account.

☐ **Import API**
Import an API from an OpenAPI definition.

☐ **Example API**
Learn about API Gateway with an example API.

API name
UserRestAPI

Description - optional

API endpoint type
Regional APIs are deployed in the current AWS Region. Edge-optimized APIs route requests to the nearest CloudFront Point of Presence. Private APIs are only accessible from VPCs.
Regional

Security policy - new | Info
Transport Layer Security (TLS) protects data in transit between a client and server. The security policy also determines the cipher suite options that clients can use with your API.
SecurityPolicy_TLS13_1_2_2021_06

Endpoint access mode - new | Info
Provide additional governance for your APIs.
☒ **Basic**
Allow all clients to access the API.

☐ **Strict (recommended)**
Enforce Server Name Indication (SNI) validation.

IP address type | Info
Select the type of IP addresses that can invoke the default endpoint for your API.
☒ **IPv4**
Supports only edge-optimized and Regional API endpoint types.

☐ **Dualstack**
Supports all API endpoint types.

Cancel **Create API**

- Keep the IP address type as **IPv4**.
- Click **Create API**.

Description - optional

API endpoint type
Regional APIs are deployed in the current AWS Region. Edge-optimized APIs route requests to the nearest CloudFront Point of Presence. Private APIs are only accessible from VPCs.
Regional

Security policy - new | Info
Transport Layer Security (TLS) protects data in transit between a client and server. The security policy also determines the cipher suite options that clients can use with your API.
SecurityPolicy_TLS13_1_2_2021_06

Endpoint access mode - new | Info
Provide additional governance for your APIs.
☒ **Basic**
Allow all clients to access the API.

☐ **Strict (recommended)**
Enforce Server Name Indication (SNI) validation.

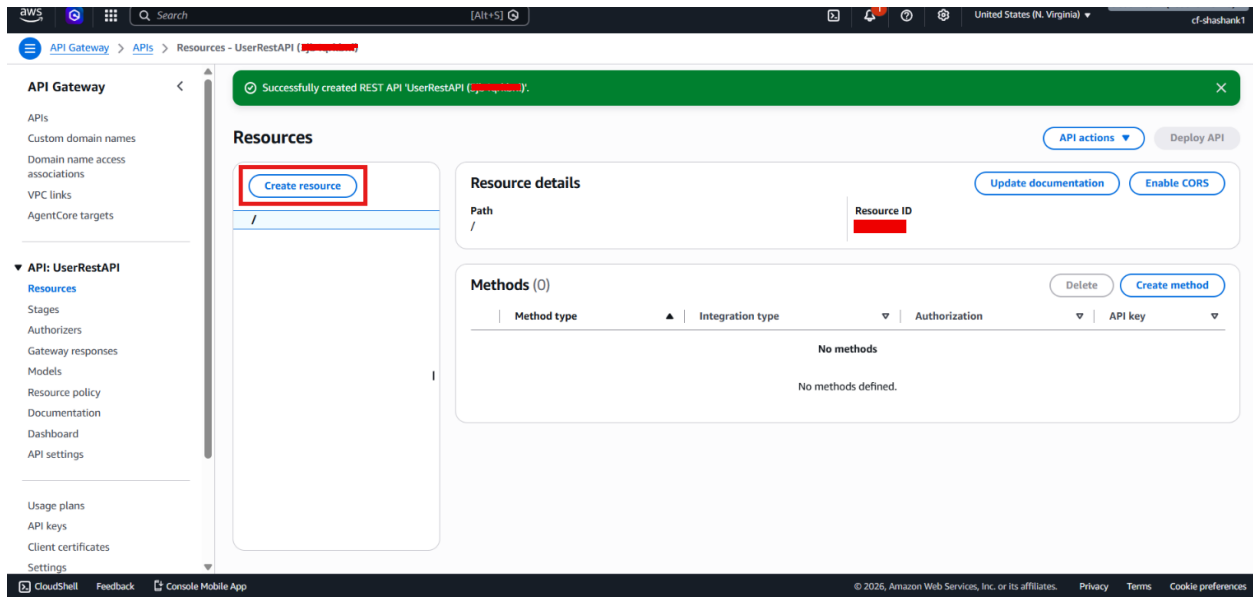
IP address type | Info
Select the type of IP addresses that can invoke the default endpoint for your API.
☒ **IPv4**
Supports only edge-optimized and Regional API endpoint types.

☐ **Dualstack**
Supports all API endpoint types.

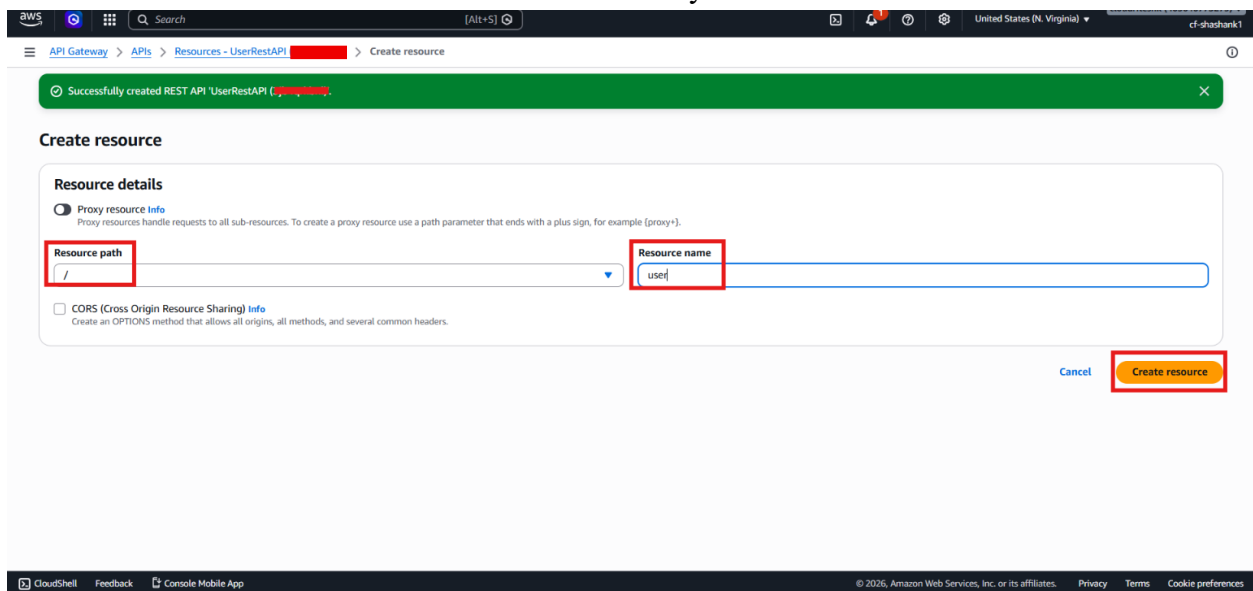
Cancel **Create API**

Step 3 – Create the Main Resource

- Open the newly created REST API.
- Click **Create Resource**.

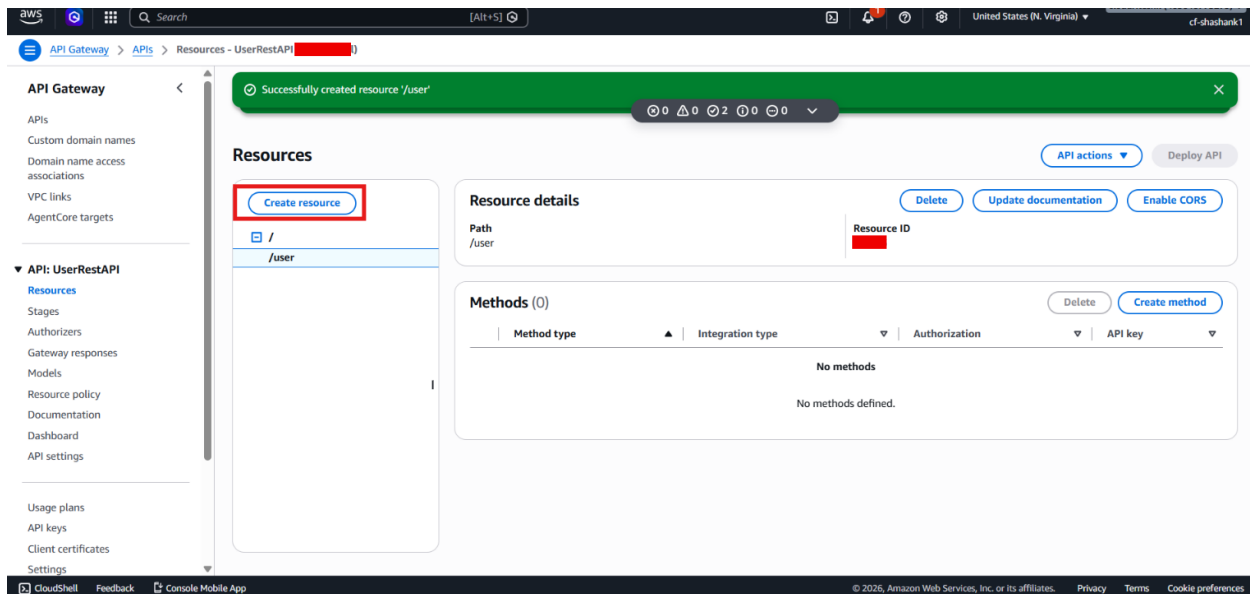


- Create the parent resource path: /user
- Confirm that the resource was created successfully.

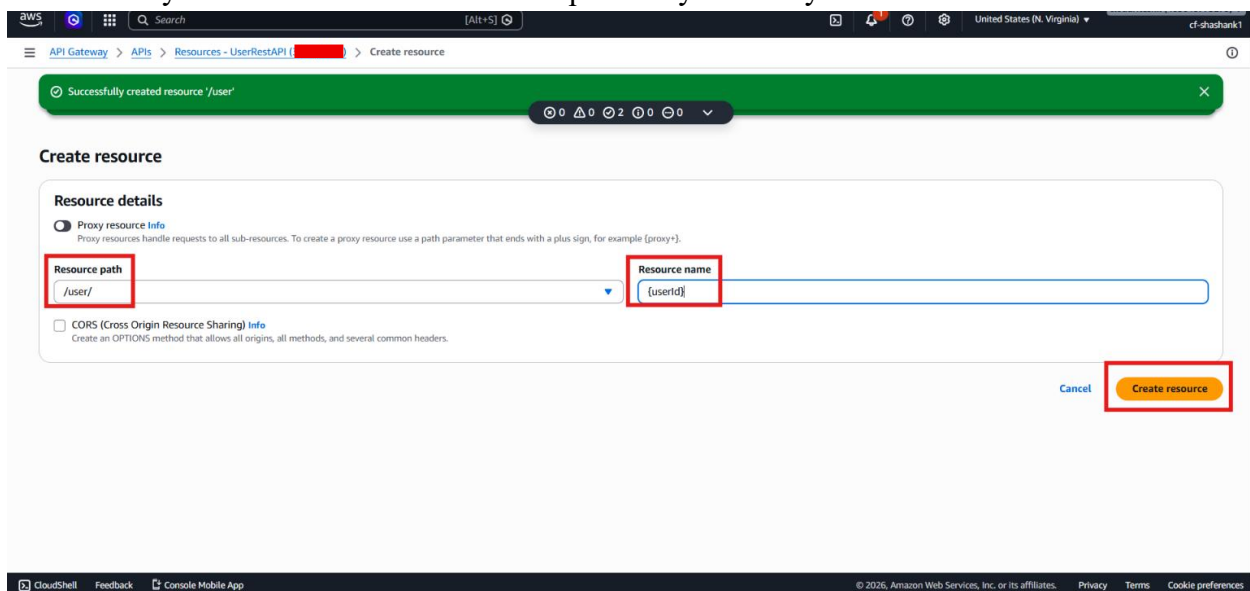


Step 4 – Create the Child Resource

- Select the /user resource.
- Click **Create Resource** again.



- Create the child resource path: `/user/{userId}`
- Use curly braces because `userId` will be passed dynamically in the URL.



- Confirm that the child resource was created successfully.

Step 5 – Prepare for Method Attachment

- Review the two resources:
 - `/user`
 - `/user/{userId}`
- Understand that each REST API URL path must be created as a separate resource.
- Prepare to attach HTTP methods in the next part of the lab.

Verification

- A **REST API** was created successfully in **API Gateway**.
- The main resource `/user` was created.
- The child resource `/user/{userId}` was created.
- The resource structure matched the API's create, read, and delete use cases.
- The REST API was ready for method creation in the next step.